

GDB 動態列印：讓你隨時隨地 printf，不需修改程式碼，不需重新編譯

原點技術於 2024-01-25 20:34:26 發佈

本文介紹了程序偵錯中的痛點，特別是 printf 的侷限性，隨後介紹了 GDB 的動態列印功能，它可以實現在不修改原始碼的情況下加入和查看程式執行時的列印資訊，極大地提高了問題定位效率。文章還詳細講解了 GDB 動態列印的實現原理和使用方法，以及如何通過保存和載入斷點資訊來解決重複設定問題。

摘要由 CSDN 通過智能技術生成

引言 - 程序偵錯的痛

偵錯是程式設計師逃脫不了的宿命！

Linux 上，有人喜歡 GDB，有人喜歡簡單粗暴的 printf。很難說哪個好，哪個不好，兩者各有各的優勢和不足，大多時候是可以互補的。

printf 最大的不足，就是需要修改程式碼，重新編譯，重新部署。而程序偵錯本身是個複雜的工作，很多時候需要不斷重複這些步驟，這就導致問題定位效率非常低。而且，當問題定位出來之後，還要花不少精力，把這些偵錯資訊全部刪掉！

對於簡單的程序，這些尚可接受。但是，在大型項目中，單是編譯建構過程可能就要幾十分鐘，甚至數個小時，而部署過程則更為複雜。你能想像得出，在這樣的項目中一直重複這些過程，是一件多麼痛苦的事情嗎？

那麼，有沒有一種方法，既不需要修改原始碼，又能隨時在程序中任何地方任意加入列印資訊呢？

當然有！GDB 的動態列印功能正式為此而生的！

GDB Dynamic Printf

GDB 提供了 Dynamic Printf 功能，下文我們稱之為動態列印。利用這個功能，我們可以在不修改程序原始碼的情況下，隨時在程序的任何地方加入格式化列印。

如此一來，當然也就不需要重新編譯和部署的過程了。

我們先看一個簡單的示例，然後再詳細介紹它的實現原理，和相關命令的用法。

示例

一個簡單示例，如下圖所示：

```
int main(int argc, char *argv[])
{
    int i, a, b;
    a = 1;
    b = 2;
    for(i = 0; i < 5; i ++) {
        a = a + i;
        b = a * 2;
    }
    return 0;
}
```

先編譯一下：

```
gcc -g test.c -o test
```

然後用 GDB 進行偵錯：

```
xxx@host:~/study/misc/gdb$ gcc -g test.c -o test
xxx@host:~/study/misc/gdb$ gdb test
Reading symbols from test...
(gdb) l 0
1      int main(int argc, char *argv[])
2      {
3          int i, a, b;
4
5          a = 1;
6          b = 2;
7
8          for(i = 0; i < 5; i++)
9          {
10             a = a + i;
11             b = a * 2;
12         }
13
14         return 0;
15     }
(gdb) r
Starting program: /home/xxx/study/misc/gdb/test
[Inferior 1 (process 17207) exited normally]
(gdb)
```

和期望的一樣，程序沒有任何列印輸出。

現在，我們在第 6 行、第 11 行、第 14 行分別加入一個動態列印斷點，用下面的命令：

```
dprintf 6,"Hello, World!\n"
dprintf 11,"i = %d, a = %d, b = %d\n",i,a,b
dprintf 14,"Leaving! Bye bye!\n"
```

如下圖：

```
(gdb) dprintf 6,"Hello, World!\n"
Dprintf 1 at 0x55555555137: file test.c, line 6.
(gdb) dprintf 11,"i = %d, a = %d, b = %d\n",i,a,b
Dprintf 2 at 0x5555555514d: file test.c, line 11.
(gdb) dprintf 14,"Leaving! Bye bye!\n"
Dprintf 3 at 0x5555555515f: file test.c, line 14.
(gdb) █
```

稍微解釋一下：

第 6 行的語句會列印一句“Hello, World!”

第 11 行會把 i、a、b 的值分別列印出來

第 14 行列印“Leaving! Bye bye!”

設定好之後，查看一下斷點的資訊：

```
(gdb) i b
Num      Type      Disp Enb Address      What
1        dprintf    keep y  0x000055555555137 in main at test.c:6
          printf "Hello, World!\n"
2        dprintf    keep y  0x00005555555514d in main at test.c:11
          printf "i = %d, a = %d, b = %d\n",i,a,b
3        dprintf    keep y  0x00005555555515f in main at test.c:14
          printf "Leaving! Bye bye!\n"
(gdb) █
```

已經設定成功了。然後，重新運行：

```
(gdb) r
Starting program: /home/xxx/study/misc/gdb/test
Hello, World!
i = 0, a = 1, b = 2
i = 1, a = 2, b = 2
i = 2, a = 4, b = 4
i = 3, a = 7, b = 8
i = 4, a = 11, b = 14
Leaving! Bye bye!
[Inferior 1 (process 17282) exited normally]
(gdb) █
```

看到了吧，儘管我們並沒有對原始碼做任何修改，且沒有重新編譯，但程序仍然按照我們的設定，列印出了我們想要的資訊！

是不是很好奇呢？GDB 的動態列印功能究竟是如何工作的呢？

GDB 動態列印實現原理

在上面的示例中，在設定好動態列印的資訊之後，我們可以用 **info break** 命令查看所設定的資訊。可見，**GDB** 的動態列印，本質上也是一種特殊的斷點。但是，它與一般的斷點又有所區別。

一般的斷點被觸發後，會中斷程序執行，然後等待使用者操作，並且使用者必須輸入 **continue** 命令讓程序恢復執行。

而動態列印斷點被觸發後，程序也會暫時中斷執行，但是不需要等待使用者響應，而是直接執行使用者預設的格式化列印語句，並自動恢復程序的執行。

GDB 動態列印的使用方法

設定動態列印的命令是 **dprintf**，格式如下：

```
dprintf location, format string, arg1, arg2, ...
```

dprintf 命令和 C 語言中的 **printf** 的用法很相似，支援格式化列印。

相比 **printf** 函數，**dprintf** 命令多了一個 **location** 參數，用於指定動態列印被觸發的位置。

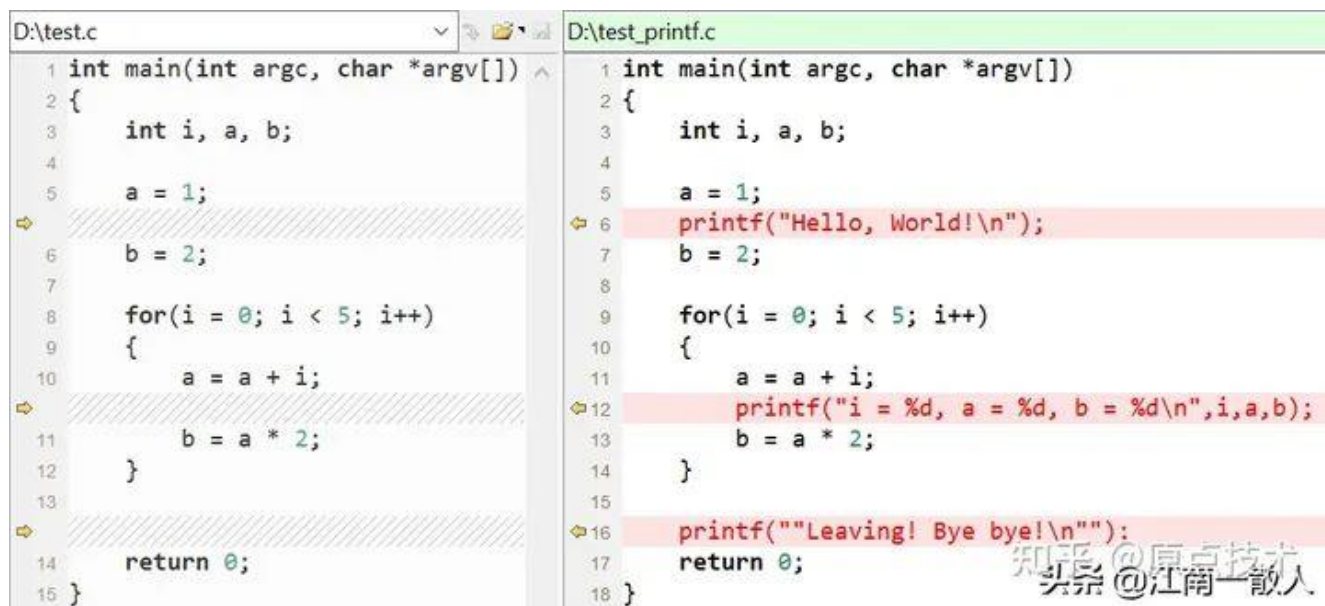
和 **break** 命令設定斷點時一樣，**location** 可以是檔案名稱:行號、函數名、或者具體的地址等。

除了 **location** 外，剩餘的幾個參數，就和 **printf()** 函數一致了。**format** 指定字串列印的格式，後面幾個參數指定列印的資料來源。

以上面示例中的命令為例：

```
dprintf 6,"Hello, World!\n"  
dprintf 11,"i = %d, a = %d, b = %d\n",i,a,b  
dprintf 14,"Leaving! Bye bye!\n"
```

在功能上等價於下圖中右側的程式碼：



```
D:\test.c
1 int main(int argc, char *argv[])
2 {
3     int i, a, b;
4
5     a = 1;
6     b = 2;
7
8     for(i = 0; i < 5; i++)
9     {
10        a = a + i;
11        b = a * 2;
12    }
13
14    return 0;
15 }

D:\test_printf.c
1 int main(int argc, char *argv[])
2 {
3     int i, a, b;
4
5     a = 1;
6     printf("Hello, World!\n");
7     b = 2;
8
9     for(i = 0; i < 5; i++)
10    {
11        a = a + i;
12        printf("i = %d, a = %d, b = %d\n", i, a, b);
13        b = a * 2;
14    }
15
16    printf("Leaving! Bye bye!\n");
17    return 0;
18 }
```

到這裡，GDB 動態列印的最基本功能就介紹完了。

斷點資訊丟失怎麼辦？

在實際項目的偵錯過程中，難免會由於各種原因而必須要反覆的偵錯才能定位出問題的原因，或者徹底理解程序的程式碼邏輯。

然而，`dprintf` 本質上也是一種斷點，因此，當偵錯結束後，本次偵錯時設定的斷點資訊就全部丟失了。如果要再次偵錯的話，就不得不重新設定一遍。

如果每次偵錯過程中，只需要設定一兩個動態列印的話，那倒也簡單。

可是，如果需要設定十幾個甚至幾十個動態列印呢？難道每次偵錯都要全部重新設定一遍嗎？想想都是一件比較麻煩的事情，對吧？

其實，GDB 也有對應的處理方案，很簡單就可以解決！

保存和載入 GDB 斷點資訊

為瞭解決上面提到的問題，GDB 很貼心地提供了對斷點資訊保存和載入的功能。

GDB 中，可以把當前所設定的各種類型的斷點資訊全部保存在一個指令碼檔案中。這其中當然也包括 `dprintf` 設定的動態列印資訊。

只需要執行下面的命令即可：

```
save breakpoints file_name
```

這條命令會把當前所有的斷點資訊都保存在 `file_name` 指定的檔案中。

等下次進行偵錯時，可以把 `file_name` 檔案中的斷點資訊重新載入起來。有兩種方法：

1. 啟動 GDB 時使用 `-x file_name` 參數。
2. 在 GDB 中執行 `source file_name` 命令。

下面分別演示一下。

保存斷點資訊

我們用 GDB 重新啟動上面示例中的 `test` 程序：

1. 用 `dprintf` 設定好斷點。
2. 用 `info break` 命令查看一下斷點資訊。
3. 執行 `save breakpoints test.bp` 命令，把斷點資訊保存在 `test.bp` 檔案中。

```
xxx@host:~/study/misc/gdb$ gdb test
Reading symbols from test...
(gdb) dprintf 6,"Hello, World!\n"
Dprintf 1 at 0x1137: file test.c, line 6.
(gdb) dprintf 11,"i = %d, a = %d, b = %d\n",i,a,b
Dprintf 2 at 0x114d: file test.c, line 11.
(gdb) dprintf 14,"Leaving! Bye bye!\n"
Dprintf 3 at 0x115f: file test.c, line 14.
(gdb) i b
Num      Type      Disp Enb Address          What
1        dprintf    keep y  0x00000000000001137 in main at test.c:6
          printf "Hello, World!\n"
2        dprintf    keep y  0x0000000000000114d in main at test.c:11
          printf "i = %d, a = %d, b = %d\n",i,a,b
3        dprintf    keep y  0x0000000000000115f in main at test.c:14
          printf "Leaving! Bye bye!\n"
(gdb) save breakpoints test.bp
Saved to file 'test.bp'.
(gdb) q
xxx@host:~/study/misc/gdb$
xxx@host:~/study/misc/gdb$ ls
test test.bp test.c
xxx@host:~/study/misc/gdb$
```

知系 @ 原点技术
头条 @ 江南一散人

我們看一下 `test.bp` 中究竟保存了什麼內容：

```
xxx@host:~/study/misc/gdb$ cat test.bp
dprintf /home/xxx/study/misc/gdb/test.c:6,"Hello, World!\n"
dprintf /home/xxx/study/misc/gdb/test.c:11,"i = %d, a = %d, b = %d\n",i,a,b
dprintf /home/xxx/study/misc/gdb/test.c:14,"Leaving! Bye bye!\n"
xxx@host:~/study/misc/gdb$
```

头条 @ 江南一散人

原來，就是我們之前執行的三條 `dprintf` 命令，並且是以文字形式存在 `test.bp` 中的。

接下來，我們用兩種方法分別載入 `test.bp` 中的斷點資訊。

用-x 參數載入斷點資訊

```
xxx@host:~/study/misc/gdb$ gdb test -x test.bp
Reading symbols from test...
Dprintf 1 at 0x1137: file test.c, line 6.
Dprintf 2 at 0x114d: file test.c, line 11.
Dprintf 3 at 0x115f: file test.c, line 14.
(gdb) i b
Num      Type      Disp Enb Address      What
1        dprintf    keep y  0x0000000000001137 in main at test.c:6
          printf "Hello, World!\n"
2        dprintf    keep y  0x000000000000114d in main at test.c:11
          printf "i = %d, a = %d, b = %d\n",i,a,b
3        dprintf    keep y  0x000000000000115f in main at test.c:14
          printf "Leaving! Bye bye!\n"
(gdb) r
Starting program: /home/xxx/study/misc/gdb/test
Hello, World!
i = 0, a = 1, b = 2
i = 1, a = 2, b = 2
i = 2, a = 4, b = 4
i = 3, a = 7, b = 8
i = 4, a = 11, b = 14
Leaving! Bye bye!
[Inferior 1 (process 18145) exited normally]
(gdb) q
xxx@host:~/study/misc/gdb$
```

知系 @ 原点技术

可見，指定-x 參數後，GDB 在開始偵錯程序之前，會從指定的檔案中把斷點資訊載入進來，並重新設定在程序中。因此，執行 run 命令後，程序能夠按照我們的預期正常執行動態列印功能。

source 命令載入斷點資訊

如下圖所示：

```

xxx@host:~/study/misc/gdb$ gdb test
Reading symbols from test...
(gdb) i b
No breakpoints or watchpoints.
(gdb) source test.bp
Dprintf 1 at 0x1137: file test.c, line 6.
Dprintf 2 at 0x114d: file test.c, line 11.
Dprintf 3 at 0x115f: file test.c, line 14.
(gdb) i b

```

Num	Type	Disp	Enb	Address	What
1	dprintf	keep	y	0x00000000000001137	in main at test.c:6
	printf "Hello, World!\n"				
2	dprintf	keep	y	0x0000000000000114d	in main at test.c:11
	printf "i = %d, a = %d, b = %d\n", i, a, b				
3	dprintf	keep	y	0x0000000000000115f	in main at test.c:14
	printf "Leaving! Bye bye!\n"				

```

(gdb) r
Starting program: /home/xxx/study/misc/gdb/test
Hello, World!
i = 0, a = 1, b = 2
i = 1, a = 2, b = 2
i = 2, a = 4, b = 4
i = 3, a = 7, b = 8
i = 4, a = 11, b = 14
Leaving! Bye bye!
[Inferior 1 (process 18164) exited normally]
(gdb) q
xxx@host:~/study/misc/gdb$

```

知乘 @ 原点技术

GDB 把 test 載入起來之後，info break 並沒有顯示出任何斷點資訊。然後，我們執行 source test.bp 命令，GDB 會把斷點資訊從 test.bp 載入進來，並重新設定在 test 程序中。

原文链接：<https://blog.csdn.net/xzh203/article/details/135853120>